Guide des scénarios S0 à S4

Fonctionnement, emplacement dans le code et validation

Carrefour Bd de l'Hôpital × Bd Saint-Marcel (Pitié-Salpêtrière, Paris 13e), simulation SUMO de 07:30 à 09:30

Comment lire ce guide. Tous les chemins partent de la racine du projet (`urban-traffic-twin/`). Une référence `fichier:ligne` donne la ligne exacte dans le code (état du commit d5f84c1) ; une modification ultérieure du fichier peut décaler les numéros. Les résultats chiffrés sont les moyennes sur les 10 graines d'évaluation, tirées de `docs/evaluation.md`. Le programme de feux est provisoire (visite terrain à venir) : les résultats le sont aussi.

Sommaire

- 1. Vue d'ensemble des 6 scénarios
- 2. Le cadre commun : de la configuration au feu affiché
- 3. Les scénarios un par un (S0, S1, S2, S2_noP, S3, S4, sonde ambulance)
- 4. La validation : pré-enregistrement, critères A-03 à A-06, statistiques
- 5. Le tableau final des résultats
- 6. Lancer et démontrer
- 7. À retenir pour la soutenance

1. Vue d'ensemble des 6 scénarios

Les 6 scénarios simulent **le même carrefour avec la même demande** de trafic (même « demand_hash ») ; seul le **contrôleur des feux** change. On peut donc comparer les scénarios graine par graine (comparaison appariée).

Code	Contrôleur	Fichier de configuration	Idée en une phrase	Retard / pers.
S0	fixed	`configs/pitie_fixed.yaml`	Feux à temps fixe : la référence.	63,6 s
S1	actuated	`configs/pitie_actuated.yaml`	Verts prolongés tant que les détecteurs voient des véhicules.	57,1 s
S2	rules	`configs/pitie_rules.yaml`	Règles : priorité bus, piétons, dégagement, priorité ambulance. **Solution principale.**	48,9 s
S2_noP	rules	`configs/pitie_rules_nopreempt.yaml`	S2 sans priorité ambulance (ablation).	48,9 s
S3	rl	`configs/pitie_rl_s3.yaml`	IA (DQN, apprentissage par renforcement) encadrée par la garde.	46,0 s
S4	rl	`configs/pitie_rl_s3_nopreempt.yaml`	L'IA S3 sans priorité ambulance (ablation).	45,9 s

Retard / pers. = `all/person_delay_mean` : retard moyen par personne, tous usagers confondus (voitures, bus pondérés par leurs passagers, piétons, ambulance). C'est le critère principal de A-06.

2. Le cadre commun : de la configuration au feu affiché

À chaque seconde simulée, le contrôleur propose une phase, la garde de sécurité la vérifie, puis SUMO l'applique. La chaîne est la même pour tous les scénarios :

Étape	Où	Ce que ça fait
Configuration de base	`configs/pitie_fixed.yaml:1-10`	07:30 (`begin_s: 27000`, l.5), 2 h de simulation (l.6), contrôleur choisi en l.8.
Les 5 phases	`configs/pitie/network.yaml:45`	HOP, S_LEFT, STM, BUS_91, BUS_57.
Choix du contrôleur	`src/traffic_twin/control/registry.py:41-48`	Lit `controller:` dans le YAML et construit la classe correspondante.

Étape	Où	Ce que ça fait
Boucle de simulation	`src/traffic_twin/sim/run_scenario.py:227-228`	`sim.step(controller.decide(sim.observe()))` à chaque seconde.
Garde de sécurité	`src/traffic_twin/sim/session.py:290` → `control/safety.py:310-328`	Vérifie chaque décision avant de l'appliquer ; aucun contrôleur ne peut la contourner.
Règles de sécurité	`configs/pitie/control.yaml:55-62`	Vert min 6 s, attente piéton ≤ 90 s (marge 2 s → 88 s), vert max 60 s, ordre des phases strict.
Résultats d'une simulation	`run_scenario.py:236-257`	Indicateurs (KPI) par usager, événements, bloc sécurité et préemption écrits dans `data/results/`.

Les règles de la garde de sécurité (`src/traffic_twin/control/safety.py`)

Règle	Rôle	Lignes
R1	Pendant une transition (jaune, rouge de dégagement), rien ne change.	`safety.py:320-321`
R2	Chaque phase garde au moins son vert minimum (6 s).	`safety.py:323-327`
R3	Priorité ambulance : chemin sûr le plus court vers la phase demandée.	`safety.py:367-389`
R4	Pas de « famine » d'une phase, ordre respecté, vert maximum.	`safety.py:331-347`
R5	Bouclier piéton : aucun piéton n'attend plus de 88 s au bord du trottoir.	`safety.py:349-365`

Chaque intervention de la garde est comptée et journalisée. Pour S0 et S1, le critère A-04 exige 0 intervention : ces contrôleurs sont sûrs par construction.

3. Les scénarios un par un

S0	Feux à temps fixe (la référence)
Idée	Chaque phase dure un temps prévu à l'avance, puis on passe à la suivante. C'est la base de toutes les comparaisons.
Configuration	`configs/pitie_fixed.yaml:8` (`controller: fixed`) ; durées des verts dans `configs/pitie/network.yaml:109` : HOP 36 s, STM 20 s, S_LEFT / BUS_91 / BUS_57 6 s, soit un cycle de 123 s.
Code	`src/traffic_twin/control/fixed.py:29-32` : on reste sur la phase tant que son vert n'est pas écoulé (l.30-31), sinon on passe à la suivante (l.32).
Résultats	Retard par personne **63,6 s** ; voitures 75,9 s de temps perdu ; bus 61,0 s de retard ; ambulance 152,2 s.

S1	Feux actionnés (déclenchés par des détecteurs)
Idée	Logique « gap-based » de SUMO : un vert est prolongé tant que des véhicules arrivent à moins de 3 s d'écart, entre un vert minimum et un vert maximum.
Configuration	`configs/pitie_actuated.yaml:5` ; paramètres dans `configs/pitie/control.yaml:19-36` : écart max 3 s, vert min 6 s, vert max HOP 38 s et STM 22 s (calculés pour que les piétons restent ≤ 88 s, sans dépendre du bouclier).
Code	`src/traffic_twin/control/actuated.py:72-82` : • l.74 : vert minimum pas encore atteint → on reste ; • l.77 : vert maximum atteint → phase suivante ; • l.82 : un véhicule détecté il y a moins de 3 s → on prolonge, sinon on passe.
Résultats	Retard par personne **57,1 s** (−10,1 %) ; voitures −13,3 % ; bus +2,5 % ; ambulance 149,1 s.

S2	Feux à règles · solution principale du projet
Idée	Base actionnée, avec des verts maximums relâchés, plus 4 règles : dégagement du carrefour, priorité bus, anticipation piétons et priorité ambulance.
Configuration	`configs/pitie_rules.yaml:4-6` + `configs/pitie/rules.yaml:27-54` : vert max HOP 50 s et STM 30 s, priorité bus sur les lignes 91, 24, 57 (prolongation jusqu'à 10 s), dégagement activé.
Code	`src/traffic_twin/control/rules.py:280-330`, dans l'ordre : 1. vert minimum (l.284) ; 2. écart entre véhicules + dégagement : on ne prolonge pas si la sortie est bloquée (l.289-298) ; 3. troncature des verts voitures pour un bus : désactivée (l.300-307 ; `rules.yaml:46`) ; 4. **priorité bus** : prolongation si le bus arrive à temps (l.309-318) ; 5. **anticipation piétons** : quitter le vert avant que le bouclier y soit obligé (l.320-326).
Priorité ambulance	Activée dans `rules.yaml:48-52`. `src/traffic_twin/control/preemption.py:158-186` gère les demandes (premier arrivé, premier servi ; abandon après 120 s sans progression). La garde mène les feux vers la phase de l'ambulance par le chemin sûr le plus court (`preemption.py:48-85`, `safety.py:367-389`).
Résultats	Retard par personne **48,9 s** (−23,1 %) ; voitures −25,4 % ; bus −8,0 % ; ambulance 110,5 s (−27,4 %) ; 10 demandes ambulance sur 10 servies, 0 abandon.

S2_noP	S2 sans priorité ambulance (ablation)
Idée	Même contrôleur que S2, priorité ambulance coupée : mesure ce qu'apporte la priorité seule.
Configuration	`configs/pitie_rules_nopreempt.yaml:4` (`preemption: {enabled: false}`).
Code	Le gestionnaire de priorité n'est pas créé : `preemption.py:122-123` renvoie None ; le reste est le code de S2.
Résultats	Retard par personne 48,9 s (identique à S2) ; ambulance **134,4 s** contre 110,5 s avec la priorité (sonde : 106,0 s contre 87,0 s).

S3	IA par apprentissage par renforcement (DQN)
Idée	Un réseau de neurones entraîné (Part 7) décide, à certains instants, de rester sur la phase ou de passer à la suivante. La garde de sécurité et la priorité ambulance gardent le dernier mot.
Configuration	`configs/pitie_rl_s3.yaml:4` : modèle `models/rl/rl7-dqn-v1-s0-415724a4/model.zip`, verrouillé par son empreinte sha256 ; hérite de `configs/pitie_rl.yaml:18-21`.
Code	`src/traffic_twin/control/rl.py` : • l.186-250 : au chargement, vérifie l'empreinte, les versions de l'observation et de la récompense ; un écart est une erreur, jamais un repli silencieux ; • l.280-296 : aux instants de décision (horloge de décision), lecture de l'état du carrefour puis `policy.predict` → 0 (rester) ou 1 (phase suivante).
Garde	Environ 29 interventions par simulation, surtout pour protéger les piétons (27,7 « ped_wait » par simulation).
Résultats	Retard par personne **46,0 s** (−27,6 %, le meilleur) ; bus −22,0 % ; ambulance 101,3 s. Contrepartie : attente moyenne des piétons +7,0 % face à S0.

S4	L'IA S3 sans priorité ambulance (ablation)
Idée	Montre comment l'IA se dégrade dans une situation jamais vue à l'entraînement (ambulance sans préemption). Ce n'est pas une solution et ce scénario n'entre jamais dans A-06.
Configuration	`configs/pitie_rl_s3_nopreempt.yaml:4` (`preemption: {enabled: false}`).
Résultats	Retard par personne 45,9 s ; ambulance **125,3 s** contre 101,3 s pour S3 (sonde : 129,5 s contre 86,5 s).

Sonde	Famille « ambulance_probe » : 7 ambulances par simulation
Idée	Mêmes contrôleurs (S0, S2, S2_noP, S3, S4), mais 7 ambulances au lieu d'une seule : la preuve la plus robuste pour l'ambulance.
Configuration	`configs/pitie_amb_*.yaml` + `configs/pitie/emergency_probe.yaml` ; déclarée dans `configs/evaluation.yaml:17-24`.
Résultats	Temps de parcours ambulance : S0 179,8 s ; S2 **87,0 s** (−51,6 %) ; S3 86,5 s ; S2_noP 106,0 s ; S4 129,5 s. 70 demandes sur 70 servies (S2 et S3).

4. La validation

4.1 Tout est fixé à l'avance (pré-enregistrement)

Les graines, les paires comparées, les indicateurs et les seuils sont écrits dans `configs/evaluation.yaml` **avant** la première simulation d'évaluation (12 octobre 2026, complété le 23 octobre pour S3 et S4). Toute modification ultérieure est datée et justifiée dans le bloc `changes`.

Élément	Lignes (evaluation.yaml)
Graines : évaluation 1-10, calibration 101-103, développement 201-207	2-3
Scénarios et paires comparées (familles main et ambulance_probe)	6-24
A-05 : priorité ambulance (test principal et tests secondaires)	25-33
A-04 : sécurité (collisions, situations dangereuses, attente piéton)	34-40
Équité : aucun usager pénalisé de plus de 5 %	41-47
A-06 : l'IA S3 contre S1, S2 et S0	50-74
Tableau final (colonnes et lignes)	75-86

4.2 La commande et les fonctions appelées

`python tasks.py evaluate` (définie dans `tasks.py:1361-1373`) lance `src/traffic_twin/evaluation/evaluate.py:2580` (`evaluate()`). Elle relance les simulations manquantes, calcule tous les critères, puis écrit `docs/evaluation.md` et `data/results/evaluation/`.

Critère	Ce qu'il vérifie	Code	Résultat
A-03	Les 4 types de contrôleur (fixe, actionné, règles, IA) ont tourné avec le même jeu d'indicateurs.	`evaluate.py:1138`	**4/4**
A-04 sécurité	Dans chaque simulation : 0 collision, 0 situation dangereuse, attente piéton au trottoir ≤ 90 s, 0 intervention de la garde pour fixe et actionné.	`evaluate.py:903`	**PASS** 110 simulations
A-05 ambulance	S2 plus rapide que S0 pour l'ambulance : Wilcoxon exact unilatéral, 10 paires, α = 0,05.	`evaluate.py:1051-1095` `evaluate.py:1109`	**PASS** 152,2 → 110,5 s p = 0,0020
A-06 IA	S3 contre S1 (principal), S2 et S0 sur le retard par personne : Wilcoxon exact bilatéral + correction de Holm. Statut procédural : jamais FAIL à cause du sens du résultat.	`final.py:140` `final.py:355`	**PASS** S3 meilleur que les trois
Équité	Aucun indicateur principal d'un usager dégradé de plus de 5 % (« dégradation cachée »).	`evaluate.py:817`	Signalée pour tous : les piétons attendent un peu plus

Les fichiers `evaluate.py` et `final.py` sont dans `src/traffic_twin/evaluation/`.

4.3 Les statistiques

Méthode	Où	Pourquoi
Test de Wilcoxon exact (apparié)	`src/traffic_twin/evaluation/stats.py:69`	Compare deux scénarios graine par graine, sans supposer une loi normale (n = 10).

Méthode	Où	Pourquoi
Estimateur de Hodges-Lehmann + IC 95 %	`stats.py:215`	Donne l'ampleur de l'écart (en secondes) et son intervalle de confiance.
Correction de Holm	`stats.py:233`	Contrôle le risque d'erreur quand on fait plusieurs comparaisons à la fois.

4.4 Les résultats de A-06 (S3 contre les références)

Comparaison	Référence	S3	Écart HL [IC 95 %]	p (Holm)	Conclusion
S3 vs S1 (principal)	57,1 s	46,0 s	−9,5 s [−17,5 ; −6,4]	0,0059	**S3 meilleur**
S3 vs S2	48,9 s	46,0 s	−2,5 s [−5,0 ; −0,6]	0,0137	**S3 meilleur**
S3 vs S0	63,6 s	46,0 s	−14,3 s [−29,4 ; −10,7]	0,0059	**S3 meilleur**

4.5 Où lire les résultats

- `docs/evaluation.md` : verdicts (l.5), tableau final (l.17-33), A-06 (l.102-106), contrôle 10.6 (l.120-127), A-04 (l.994-1008), A-05 (l.1014).
- `data/results/evaluation/` : `a05.json`, `a06.json`, `final_table.csv`, `summary.json` (lus par l'API, l'application et le rapport).
- `docs/figures/pitie/eval/` : les figures (par exemple `a06_forest.png`).

5. Le tableau final des résultats

Moyenne ± écart-type sur les 10 graines d'évaluation ; entre parenthèses, l'écart par rapport à S0. Source : `docs/evaluation.md:17-33`.

Usager	Indicateur	S0	S1	S2	S3	S2_noP	S4
Voiture	Temps perdu moyen (s)	75,9	65,8 (−13,3 %)	56,6 (−25,4 %)	56,2 (−25,9 %)	55,9 (−26,3 %)	56,1 (−26,1 %)
Voiture	Attente moyenne (s)	60,9	52,0 (−14,7 %)	44,2 (−27,5 %)	44,2 (−27,5 %)	43,5 (−28,6 %)	44,0 (−27,7 %)
Voiture	Débit (véh/h)	1178,8	1179,5 (+0,1 %)	1185,7 (+0,6 %)	1185,7 (+0,6 %)	1184,7 (+0,5 %)	1185,3 (+0,6 %)
Bus	Retard moyen (s)	61,0	62,6 (+2,5 %)	56,1 (−8,0 %)	47,6 (−22,0 %)	57,2 (−6,3 %)	47,4 (−22,3 %)
Piéton	Attente moyenne au trottoir (s)	18,9	19,2 (+1,8 %)	19,4 (+2,6 %)	20,2 (+7,0 %)	19,1 (+1,3 %)	20,1 (+6,6 %)
Piéton	Attente maximale (s), limite 90	85,6	87,4	88,0	88,0	88,0	88,0
Ambulance	Temps de parcours (s)	152,2	149,1 (−2,0 %)	110,5 (−27,4 %)	101,3 (−33,4 %)	134,4 (−11,7 %)	125,3 (−17,7 %)
Tous	**Retard par personne (s)**	**63,6**	**57,1 (−10,1 %)**	**48,9 (−23,1 %)**	**46,0 (−27,6 %)**	**48,9 (−23,0 %)**	**45,9 (−27,8 %)**

Sécurité (A-04) : 0 collision et 0 situation dangereuse dans les 6 scénarios ; attente piéton maximale ≤ 88 s partout. Interventions de la garde par simulation : 0 pour S0, S1, S2 et S2_noP ; 29 pour S3 ; 30 pour S4.

6. Lancer et démontrer

Toutes les commandes se lancent à la racine du projet, avec l'environnement Python du projet (`.venv`).

Lancer un scénario seul (exemple : S2, graine 201)

```
python -m traffic_twin.sim.run_scenario --config configs/pitie_rules.yaml --seed 201
```

Ajouter `--gui` pour voir la simulation dans SUMO. Point d'entrée : `src/traffic_twin/sim/run_scenario.py:260`. Remplacer le fichier de configuration pour un autre scénario (tableau de la section 1). Les graines 1 à 10 sont réservées à l'évaluation officielle et protégées par un verrou dans `tasks.py`.

Refaire toute l'évaluation et ses figures

```
python tasks.py evaluate
python tasks.py plots-eval
```

Montrer les résultats dans l'application (API + interface)

```
python tasks.py demo
```

Puis ouvrir http://127.0.0.1:8000/mock : le menu « Scénario · graine » choisit le scénario rejoué ; le cockpit admin affiche le tableau comparatif et la comparaison synchronisée S1 à S4.

7. À retenir pour la soutenance

- **Même carrefour, même demande, seul le contrôleur change** : la comparaison graine par graine (10 paires) est donc équitable.
- **La sécurité ne dépend pas du contrôleur** : la garde (`control/safety.py`) vérifie chaque décision ; A-04 est PASS sur les 110 simulations.
- **S2 tient sa promesse pour l'ambulance** : A-05 PASS (152,2 → 110,5 s, p = 0,0020) ; avec 7 ambulances, −51,6 %.
- **S3 (l'IA) obtient le meilleur score global** (46,0 s, meilleur que S1, S2 et S0 dans A-06).
- **Mais S2 reste la solution principale** : c'était une règle fixée avant les essais (`configs/evaluation.yaml:73`), et S3 échoue au contrôle C2 : l'attente moyenne des piétons est 5,2 % plus longue qu'avec S1 (`docs/evaluation.md:123`).
- **Honnêteté** : données simulées, programme de feux supposé jusqu'à la visite terrain ; tout écart au plan est daté dans `configs/evaluation.yaml` (bloc `changes`).